UCS503-SOFTWARE ENGINEERING

CityServe

A Local City Services Marketplace for Patiala

1024030685 Ruhanika

1024030686 Ayushi

1024030683 Karan Bansal

Team Cyphers Group: 3C51 BE Third Year, CSE

Advisor: Raghav B. Venkataramaiyer

Project Report

Computer Science and Engineering Department

Thapar Institute of Engineering and Technology, Patiala

Contents

Chapter 1

Introduction

1.1 Background and Context

Patiala, home to a large student population associated with the Thapar Institute of Engineering and Technology, hosts a wide range of local businesses – grocery stores, tiffin services, dairy suppliers, electricians, plumbers, tutors, and laundry providers – that operate largely without any shared digital storefront. Customers typically discover these vendors through word of mouth, printed pamphlets, or informal messaging groups, while vendors have no structured way to receive, track, or manage incoming orders. CityServe was developed to address this gap by providing a single web application through which students and residents can discover local products and services, while allowing vendors to manage their offerings and incoming orders from one dashboard.

1.1.1 Problem Statement

There is no unified platform through which a resident of Patiala can browse both products (groceries, tiffins, dairy items) and services (electricians, plumbers, tutors, laundry) offered by local vendors, place an order, track its status, and leave feedback once it is fulfilled. Vendors similarly lack a lightweight system to list what they offer, receive orders, and communicate status updates back to customers without relying on ad-hoc phone calls or messages.

1.1.2 Motivation

The project is motivated by the observation that most local-commerce platforms are built for large-scale e-commerce and are unnecessarily complex for a campus-town setting. A large fraction of local transactions in Patiala are either simple purchases or simple service bookings, and both categories share nearly the same lifecycle: a customer requests something, a vendor confirms or declines it, and the transaction is eventually marked complete. This observation directly shaped the design of CityServe, which represents products and services – and the orders and bookings that result from them – as a single, unified concept rather than as separate parallel systems.

1.1.3 Project Objectives

1. To design and implement a role-based web application supporting customers, vendors, and administrators.

2. To provide a unified data model in which both physical products and time-based services can be listed, ordered, and reviewed through the same underlying structures.

3. To implement a clear, server-validated order lifecycle so that order status can never be changed to an invalid state from the client side.

4. To build a secure authentication system using JSON Web Tokens with refresh-token rotation, and to enforce ownership-based access control on all sensitive operations.

5. To integrate a real, location-based vendor dataset for Patiala so that the platform reflects an actual local business ecosystem rather than only synthetic demo data.

1.1.4 Scope of the Project

The scope of CityServe covers the complete customer-to-vendor order lifecycle: account registration and authentication for customers and vendors, category and offering browsing, order placement and tracking, vendor-side order management, a rating and review system restricted to completed orders, and an administrative layer for vendor verification and platform-level analytics. Payment processing and map-based location lookup are integrated at the interface level, with a documented demonstration fallback used when third-party API credentials are not configured, so that the application remains fully usable in a development or evaluation environment.

Chapter 2

Methodology and Design

2.1 System Architecture

CityServe follows a layered client-server architecture. The frontend is a Next.js application (App Router) [1] that renders customer-, vendor-, and admin-facing pages and communicates with a separate Express [3] backend over a REST API. The backend is written in TypeScript [2] and uses Sequelize [4] as its Object-Relational Mapper on top of a MySQL [5] database. Figure 2.1 shows the overall structure of the system.

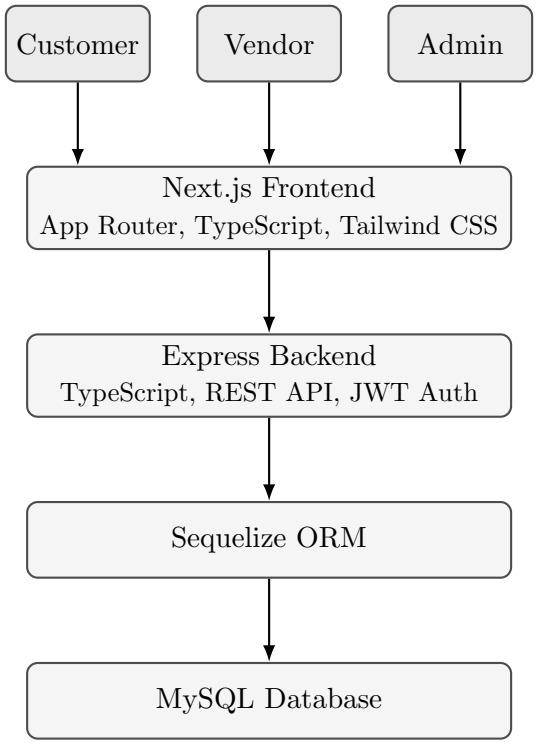

Figure 2.1: High-level system architecture of CityServe.

2.1.1 High-Level Design

Requests from the browser are handled entirely by the Next.js frontend, which renders pages for browsing categories and offerings, managing a client-side cart, viewing and

tracking orders, and, for vendors and administrators, managing offerings, orders, and platform data respectively. All data-modifying and data-fetching operations are routed through a versioned REST API exposed by the Express backend. The backend authenticates each request using a JWT access token, applies role-specific authorization checks, and uses Sequelize models to read from and write to the underlying MySQL database.

2.2 User Roles

CityServe defines exactly three user roles, distinguished by a `role` column on the `users` table. Table 2.1 summarizes the responsibilities of each role.

Role	Responsibilities
Customer	Browse categories and offerings, place orders, cancel pending orders, track order status, submit ratings and reviews for completed orders, manage personal profile.
Vendor	Maintain a vendor business profile, list products and services, review and approve or reject incoming orders, mark approved orders as completed, view all orders placed against the vendor account.
Admin	View and manage the platform's users and vendors, verify or unverify vendor accounts, and view platform-level analytics.

Table 2.1: User roles and their responsibilities.

2.3 Core Workflow

Every transaction on CityServe, whether it is a product purchase or a service request, follows the same order lifecycle. A vendor lists a product or service under a category. A customer browses this offering and places an order, which is created in the `PENDING` state. The vendor then either approves or rejects the order. An approved order is fulfilled by the vendor and subsequently marked `COMPLETED`, at which point the customer may submit a rating and an optional written review. A customer may also cancel an order while it remains `PENDING`. Figure 2.2 illustrates the resulting state machine.

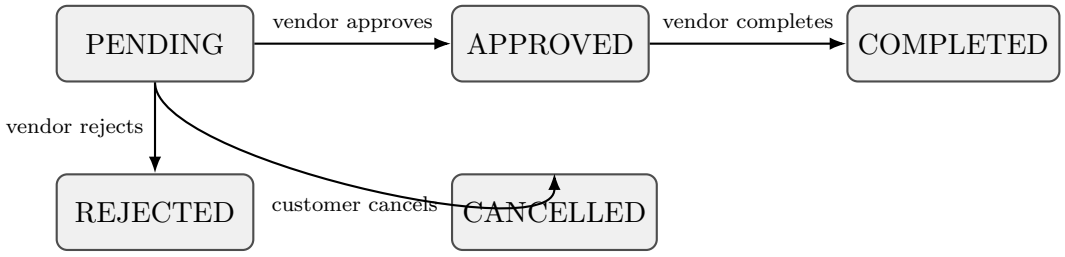

Figure 2.2: Order status state machine used by CityServe.

2.4 Database Design

CityServe's persistence layer consists of six logical tables, described in Table 2.2. The schema is intentionally compact: rather than modelling products and services, or purchases and bookings, as separate concepts, CityServe treats them as specialisations of two general concepts – an `offering` and an `order`.

Table	Purpose
`users`	Stores customer, vendor, and admin accounts. A `role` column (`CUSTOMER` / `VENDOR` / `ADMIN`) distinguishes account types.
`categories`	Stores the product and service categories under which offerings are organised (for example, Electrical Services or Tutoring and Coaching).
`vendors`	Stores vendor business profiles – business name, description, address, city, coordinates, and verification status – linked one-to-one with a `VENDOR`-role user.
`offerings`	Stores every product and service listed on the platform. A `type` column (`PRODUCT` / `SERVICE`) distinguishes the two, with optional type-specific fields such as `stock` and `unit` for products, and `durationMinutes` for services.
`orders`	Stores every customer transaction, whether it represents a product purchase or a service request. Service-oriented orders additionally carry a `scheduledAt` timestamp; product orders carry quantity-related information.
`reviews`	Stores at most one review per completed order, consisting of a 1–5 star rating and an optional text comment.

Table 2.2: The six logical tables that make up the CityServe database.

To avoid duplication between products and services, CityServe represents both using a single unified `offerings` table, with the `type` attribute distinguishing `PRODUCT` from `SERVICE`. A product and a service are, from the platform's point of view, the same underlying concept: something a vendor offers, at a price, under a category, that a customer can order. Modelling them as one table means that browsing, searching, and displaying offerings only requires a single code path, and that adding a new offering type in future would not require a schema change.

The same reasoning applies to orders. CityServe uses a unified `orders` table to represent both product purchases and service requests. A service request is simply an order whose associated offering has `type = SERVICE`, with scheduling information captured in the `scheduledAt` attribute; a product purchase is an order against a `PRODUCT` offering. Keeping both in a single table means that the order status workflow, the vendor order dashboard, the review system, and the ownership and authorization checks described in Section 2.11 are each implemented once and apply uniformly across the entire platform, rather than being duplicated across parallel product and service subsystems.

2.5 Entity Relationships

The relationships between the six tables are strictly one-to-many or one-to-one; the schema contains no many-to-many relationships. Table 2.3 lists every relationship, and

Figure 2.3 presents the same information as an entity diagram.

Entity	Cardinality	Entity
User	1 – 0/1	Vendor
Category	1 – N	Vendor
Category	1 – N	Offering
Vendor	1 – N	Offering
User (as customer)	1 – N	Order
Vendor	1 – N	Order
Offering	1 – N	Order
Order	1 – 0/1	Review
User (as customer)	1 – N	Review
Vendor	1 – N	Review

Table 2.3: Entity relationships in the CityServe schema.

2.6 Technical Stack

2.7 Technical Stack

CityServe is built entirely on a TypeScript-based stack across both the frontend and backend. The frontend uses Next.js [1], TypeScript [2], and Tailwind CSS [9], while the backend uses Express [3]. The database layer uses MySQL [5] through the Sequelize ORM [4]. The complete technology stack is summarised in Table 2.4.

Layer	Technology
Frontend	Next.js (App Router), TypeScript, Tailwind CSS
Backend	Express, TypeScript
Database	MySQL, accessed through the Sequelize ORM
Authentication	JWT access tokens and refresh tokens, with refresh tokens stored in HttpOnly cookies
Maps	Mapbox, with a demonstration/fallback location used when a Mapbox API key is not configured
Payments	Razorpay, with a demonstration payment flow used when a Razorpay API key is not configured
Icons	lucide-react

Table 2.4: Technology stack used in CityServe.

2.8 REST API Design

CityServe exposes a resource-oriented REST API. Table 2.5 lists every endpoint grouped by resource.

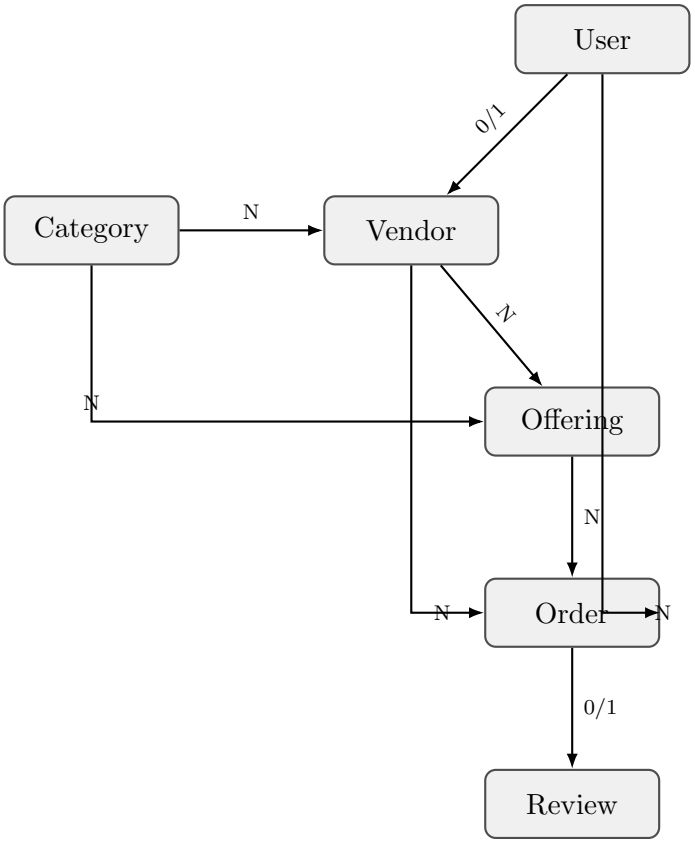

Figure 2.3: Entity relationship diagram for the CityServe schema.

2.9 Authentication and Authorization

CityServe uses a two-token JWT scheme [6]. On successful login or registration, the backend issues a short-lived access token, which the frontend attaches to subsequent API requests, and a longer-lived refresh token, which is stored in an HttpOnly cookie so that it is never directly accessible to frontend JavaScript. When the access token expires, the frontend calls `POST /auth/refresh`, which validates the refresh cookie and issues a new access token. Every protected route on the backend verifies the access token and reads the requester's `id` and `role` from it; role-specific routes (vendor and admin routes in particular) additionally check that the authenticated user holds the required role before proceeding.

2.10 Implementation Details

2.10.1 Cart and Checkout

Because CityServe does not persist a server-side shopping cart, the customer-facing cart is implemented entirely on the client. Selected offerings are held in the browser's `localStorage` so that the cart survives page reloads and navigation. At checkout, the frontend uses the customer's stored profile address – which can be edited inline before confirming – and creates one order per checkout through `POST /orders`. Service offerings are ordered through the same endpoint, with an additional `scheduledAt` field capturing the time at which the service is requested, so that both product purchases and service

requests are created through a single, consistent code path.

2.10.2 Vendor Offering and Order Management

Vendors manage their offerings through the `offerings` endpoints and view incoming orders through `GET /vendors/my/orders`, which returns each order together with the associated offering and customer information needed to fulfil it. A vendor updates an order's status through `PATCH /vendors/my/orders/:id/status`, which is validated server-side as described in Section 2.11.

2.10.3 Data Sourcing

CityServe's vendor directory is seeded with a dataset of 155 Patiala businesses sourced through the Google Places API, covering a mix of product vendors and service providers across the categories supported by the platform. Vendors that originate from this imported dataset are marked as unverified by default; verification is a separate administrative action performed through `PATCH /admin/vendors/:id/verify` once a vendor's details have been confirmed. Presenting imported businesses as unverified until explicitly verified avoids implying that every listed business has been independently confirmed by the platform, while still allowing the directory to reflect a realistic set of local vendors from the outset.

2.10.4 Order Status Badging

The customer- and vendor-facing interfaces display an order's current status using a badge component that distinguishes all five possible states – `PENDING`, `APPROVED`, `COMPLETED`, `REJECTED`, and `CANCELLED` – with a distinct visual treatment for each, so that the state of any order is immediately clear to both parties.

2.11 Security Considerations

Table 2.6 summarizes the security mechanisms implemented in CityServe; each is described in more detail below.

Ownership and role scoping are enforced directly inside the backend controllers rather than relied upon at the client: every handler that reads or writes an order re-derives the requester's identity from the verified access token and filters or authorizes the operation accordingly, so that a customer or vendor cannot act on records that do not belong to them regardless of what identifiers are supplied in the request.

Order status transitions are restricted to the following valid paths, matching the state machine shown in Figure 2.2:

- `PENDING` → `APPROVED`
- `PENDING` → `REJECTED`
- `PENDING` → `CANCELLED`
- `APPROVED` → `COMPLETED`

In particular, a `PENDING` order can never transition directly to `COMPLETED`; it must first be approved by the vendor. This prevents an order from being marked fulfilled before the vendor has had the opportunity to confirm it.

Resource	Endpoints

Authentication	`POST /auth/register/customer`
	`POST /auth/register/vendor`
	`POST /auth/login`
	`POST /auth/logout`
	`POST /auth/refresh`
	`GET /auth/me`
	`PATCH /auth/me`

Categories	`GET /categories` `GET /categories/:slug`

Catalog	`GET /catalog/products`
	`GET /catalog/services`
	`GET /catalog/products/:id`
	`GET /catalog/services/:id`
	`POST /offerings`
	`PUT /offerings`

Vendors	`GET /vendors`
	`GET /vendors/:id`
	`GET /vendors/my/orders`
	`PATCH /vendors/my/orders/:id/status`

Orders	`POST /orders`
	`GET /orders`
	`GET /orders/:id`

Reviews	`POST /reviews`
	`GET /reviews`
	`GET /reviews/vendor/:vendorId`

Search	`GET /search?q=`

Admin	`GET /admin/users`
	`GET /admin/vendors`
	`PATCH /admin/vendors/:id/verify`
	`GET /admin/analytics`

Table 2.5: REST API endpoints exposed by the CityServe backend.

Mechanism	Description
Ownership scoping	Every order query for a customer is scoped to that customer's own `id`; a customer cannot view or modify another customer's order simply by guessing its identifier.
Vendor scoping	Vendor operations (viewing and updating orders, managing offerings) are scoped to the authenticated vendor's own account.
Status transition validation	Order status changes are validated on the server against an explicit allow-list of transitions, independent of anything enforced in the user interface.
Review restrictions	A review may be created only for an order in the `COMPLETED` state, only by the customer who placed that order, and at most once per order.
Password hashing	Passwords are hashed using bcrypt before storage and are never returned in any API response.
Refresh token storage	Refresh tokens are stored in HttpOnly cookies, preventing them from being read by frontend JavaScript.

Table 2.6: Security controls implemented in CityServe.

Chapter 3

Results and Evaluation

3.1 Testing Methodology

CityServe was evaluated using a combination of static verification, functional testing, and integration testing across the frontend and backend. Static verification confirmed that both codebases type-check correctly; functional testing exercised each role's core operations against the REST API; and integration testing confirmed that the frontend, backend, and database work correctly together end to end, including a full production build of the frontend application.

3.2 Integration Testing

Integration testing focused on verifying that data flows correctly across all three layers of the system: that offerings created by a vendor are correctly displayed to customers with accurate vendor and category information attached, that placing an order through the frontend results in a correctly persisted row in the `orders` table, and that a vendor's dashboard reflects the customer and offering details for every order assigned to that vendor. Authentication integration was verified by confirming that the access-token and refresh-token cycle correctly restores an authenticated session after the access token expires, using the HttpOnly refresh cookie.

3.3 Functional Testing

Functional testing was organised around each role's set of operations, summarized in Table 3.1.

Table 3.2 lists representative test cases used during functional testing. Expected outcomes are phrased as expected behaviour of the implemented system; execution results reflect functional verification performed against the running application rather than automated numerical benchmarking.

3.4 Build and Compilation Verification

The TypeScript compiler was run with the `tsc --noEmit` flag on both the backend and frontend codebases, and both completed without type errors. A complete Next.js pro-

Role	Verified Functionality
Customer	Browse offerings, place orders, cancel pending orders, view order history and status, submit reviews for completed orders, update profile information.
Vendor	Manage offerings, view incoming orders with full customer and offering detail, approve or reject pending orders, mark approved orders as completed, manage vendor profile information.
Admin	View platform users, view vendors, verify vendor accounts, view platform analytics.

Table 3.1: Role-wise functionality verified during testing.

Scenario	Expected Behaviour	Result
Customer places an order for a product offering	Order is created with status `PENDING`	Verified
Customer attempts to cancel an `APPROVED` order	Request is rejected; only `PENDING` orders can be cancelled	Verified
Vendor approves a `PENDING` order	Order transitions to `APPROVED`	Verified
Vendor attempts to set a `PENDING` order directly to `COMPLETED`	Request is rejected by server-side transition validation	Verified
Customer submits a review for a `COMPLETED` order	Review is created and linked to the order	Verified
Customer attempts a second review on the same order	Request is rejected; only one review is permitted per order	Verified
Customer attempts to view another customer's order by identifier	Request is rejected; access is scoped to the authenticated customer	Verified
Admin verifies a vendor account	Vendor's verification status is updated and reflected in listings	Verified

Table 3.2: Representative test cases exercised during functional testing.

duction build was subsequently executed for the frontend application; every route in the application, including every customer-, vendor-, and admin-facing page, built successfully without runtime errors during static generation. Together, these checks confirm that the codebase is internally type-consistent and that the frontend can be built into a deployable production bundle.

3.5 Issue Identification and Resolution

During functional and integration testing, several implementation issues were identified and resolved prior to final verification. Table 3.3 summarizes representative issues and their resolutions.

Issue Identified	Resolution
Offering listings did not display vendor and category information	Explicit lowercase association aliases were added to the relevant Sequelize associations so that offering, vendor, and category data is consistently keyed and correctly returned to the frontend.
Vendor order listings did not include offering and customer detail	The required associations were added to the vendor order query so that each order returned to a vendor includes the associated offering and customer information needed to fulfil it.
Order status could be set to an arbitrary value from the client	Server-side validation of status transitions was added, restricting updates to the valid paths described in Section 2.11.
Vendor registration did not persist business address, city, coordinates, or description	The registration handler was corrected to read and persist the complete business profile submitted by the frontend.
Order status indicators did not visually distinguish all order states	The status badge component was updated to represent all five order statuses distinctly.

Table 3.3: Representative issues identified and resolved during testing.

3.6 Performance and System Evaluation

Formal, numerical performance benchmarking – such as measured response times, request throughput, or resource utilization under load – was not carried out as part of this project and no such figures are reported here. System evaluation instead relied on functional, integration, and build verification as described above, which confirm that the application behaves correctly across its supported workflows and compiles and builds cleanly for deployment.

3.7 Results and Analysis

The testing process confirmed that CityServe meets the objectives set out in Chapter 1: customers, vendors, and administrators can each complete their respective workflows end to end; the order status machine cannot be driven into an invalid state from the client; reviews are correctly restricted to completed orders and to the customer who placed them; and the application builds successfully as a deployable Next.js production bundle. The unified `offerings` and `orders` design was validated in practice, since both product and service workflows were exercised through the same endpoints and the same status machine without requiring any offering- or order-type specific branching in the security or ownership checks.

Chapter 4

Conclusion

4.1 Summary of Work

CityServe was designed and implemented as a role-based local-commerce platform for Patiala, built on a Next.js frontend and an Express and MySQL backend. The system supports customer, vendor, and admin roles over a compact six-table schema in which products and services share a single `offerings` table and purchases and service requests share a single `orders` table, connected by a well-defined, server-validated status workflow. Authentication is handled through JWT access and refresh tokens, with refresh tokens held in HttpOnly cookies, and the platform is seeded with a real dataset of Patiala businesses.

4.2 Key Findings

- Representing products and services as a single `type`-tagged offering, and purchases and bookings as a single order, allowed the review system, the order workflow, and the ownership-based authorization checks to be implemented once and applied uniformly across the whole platform.
- Server-side validation of order status transitions is essential even when the user interface already restricts which actions are presented to the user, since it is the only reliable guarantee against invalid state changes.
- Associations returned by an ORM must be explicitly and consistently aliased; without explicit aliases, related data can be silently omitted from API responses without raising any error, making such issues easy to miss without close functional testing.

4.3 Limitations

- Payment processing falls back to a demonstration flow when Razorpay API credentials are not configured, rather than always processing a real transaction.
- Map-based location features fall back to a demonstration location when a Mapbox API key is not configured.

- Imported vendor records are initially unverified until confirmed through the administrative verification action, so not every listed business has necessarily been independently verified at any given time.
- There is currently no dedicated notification system; customers and vendors must check the platform directly for status updates.
- Address management is intentionally simplified to a single profile address per customer rather than a full address book.

4.4 Future Work and Recommendations

1. Introduce pagination for vendor and offering listings to support larger datasets more efficiently.
2. Add a proper address book allowing repeat customers to store and select from multiple delivery addresses.
3. Replace the demonstration payment flow with full, live Razorpay integration.
4. Support image upload for offerings, rather than relying solely on externally hosted images.
5. Introduce a notification system to alert customers and vendors of order status changes without requiring them to check the platform directly.

4.5 Final Remarks

CityServe demonstrates that a local-commerce platform serving a mixed product-and-service marketplace does not require separate parallel systems for each category of offering. By treating products and services, and purchases and bookings, as specializations of two shared concepts, the platform achieves a small, consistent schema and a single set of workflow and security rules that apply uniformly across the entire system, while still supporting the full range of interactions expected of a real local-services marketplace.

Bibliography

[1] Vercel. *Next.js Documentation*. 2024. Available at: https://nextjs.org/docs

[2] Microsoft. *TypeScript Documentation*. 2024. Available at: https://www.typescriptlang.org/docs/

[3] OpenJS Foundation. *Express – Node.js Web Application Framework*. 2024. Available at: https://expressjs.com/

[4] Sequelize Contributors. *Sequelize – Feature-rich ORM for Modern Node.js and TypeScript*. 2024. Available at: https://sequelize.org/

[5] Oracle Corporation. *MySQL Reference Manual*. 2024. Available at: https://dev.mysql.com/doc/

[6] Michael B. Jones, John Bradley, and Nat Sakimura. *JSON Web Token (JWT)*. Internet Engineering Task Force, RFC 7519, 2015. Available at: https://www.rfc-editor.org/rfc/rfc7519

[7] Mapbox. *Mapbox Documentation*. 2024. Available at: https://docs.mapbox.com/

[8] Razorpay. *Razorpay Developer Documentation*. 2024. Available at: https://razorpay.com/docs/

[9] Tailwind Labs. *Tailwind CSS Documentation*. 2024. Available at: https://tailwindcss.com/docs